Unit-III

Unit–III:	No. of Lectures: 08 Hours	Marks: 12
Evaluating machine learning algorithms and model selection: Machine Learning Algorithms, Designing Machine Learning Algorithms, Classification Metrics Regression Metrics, Statistical Learning Theory, Ensemble Methods, What is Random Forest **Sparse modeling and estimation**: Time series, Deep (Structured) Learning, Neural Network, Applications of Deep Learning Methods, Feature Representation Learning.		

Evaluating machine learning algorithms and model selection

- Machine Learning Algorithms
- Designing Machine Learning Algorithms
- Classification Metrics
- Regression Metrics
- Statistical Learning Theory
- Ensemble Methods
- What is Random Forest?

Machine Learning Algorithms

- As more and more decisions in today's world are being based on machine learning models, it's imperative for us to learn how to diagnose these models before they fail in production.
- Machine learning has become a central part of our life – as consumers, customers, and hopefully as researchers and practitioners.
- Whether we are applying predictive modeling techniques to our research or business problems.
- Here one thing is common: Everyone want to make "good" predictions!
- Fitting a model to our training data is one thing, but how do we know that it generalizes well to unseen data?
- And how do we select a good model? Maybe a different learning algorithm could be better-suited for the given problem?

Machine Learning Algorithms

- In data science mainly three steps are involved:
  - Data Preparation, data munging and process algorithms
  - Optimization of algorithms for parameter estimation.
  - Machine Learning Algorithms.
- Once the problem is defined and data is prepared, there is a need to apply machine learning algorithms to the data in order to solve the problem.
- A lot of time can be spend on choosing, running and tuning algorithms. Here you must make sure that you are using a time effectively to get closer to your goal.

Designing Machine Learning Algorithms

- Test Harness
- Performance Measure
- Test and Train Datasets
- Cross Validation
- Testing Algorithms

Test Harness

- The test harness is the data that user will train and test an algorithm against it.
- The performance measure is used to assess its performance.
- It is important to define a test harness well, so that user can focus on evaluating different algorithms and thinking deeply about the problem.
- The goal of the test harness is to be able to quickly and consistently test algorithms against a fair representation of the problem being solved.
- The outcome of testing multiple algorithms against the harness will be an estimation of how a variety of algorithms perform on the problem against a chosen performance measure.

Test Harness

- User will know which algorithms might be worth tuning on the problem and which should not be considered further.
- The results will also give you an indication of how learnable the problem is.
- If a variety of different learning algorithms universally perform poorly on the problem, it may be an indication of a lack of structure available to algorithms to learn.
- This may be because there is actually a lack of learnable structure in the selected data or it may be an opportunity to try different transforms to expose the structure to the learning algorithms.

Performance Measure

- The performance measure is the way to evaluate a solution to the problem. It is the measurement to make of the predictions made by a trained model on the test dataset.
- Performance measures are typically specialized to the class of problem user are working with, for example classification, regression, and clustering.
- Many standard performance measures will give a score that is meaningful to problem domain.
- For example, classification accuracy for classification (total correct correction divided by the total predictions made multiple by 100 to turn it into a percentage).

Performance Measure

- If anyone wants a more detailed breakdown of performance, for example, one may need to know about the false positives on a spam classification problem because good email will be marked as spam and cannot be read.
- There are many standard performance measures to choose from.
- No need to plan a new performance measure, as user can generally find or adapt one that best captures the requirements of the problem being solved.
- Looking to similar problems that are uncovered and at the performance measures used to see if any can be adopted.

Test and Train Datasets

- From the transformed data, you will need to select a test set and a training set.
- An algorithm will be trained on the training dataset and will be evaluated against the test set.
- This may be as simple as selecting a random split of data (66% for training, 34% for testing) or may involve more complicated sampling methods.
- A trained model is not exposed to the test dataset during training and any predictions made on that dataset are designed to be indicative of the performance of the model in general.
- As such one must make sure, the selection of the datasets are representative of the problem we are solving.

Cross Validation

- A more sophisticated approach than using a test and train dataset is to use the entire transformed dataset to train and test a given algorithm.
- A method that could be used in test harness which does this is called **cross validation**.
- It first involves separating the dataset into a number of equally sized groups of instances (called folds).
- The model is then trained on all folds exception one that was left out and the prepared model is tested on that left out fold.
- The process is repeated so that each fold get's an opportunity at being left out and acting as the test dataset.
- Finally, the performance measures are averaged across all folds to estimate the capability of the algorithm on the problem.

Cross Validation

- The number of folds can vary based on the size of your dataset, but common numbers are 3, 5, 7 and 10 folds. The goal is to have a good balance between the size and representation of data in your train and test sets.
- When you're just getting started, stick with a simple split of train and test data (such as 66%/34%) and move onto cross validation once you have more confidence.

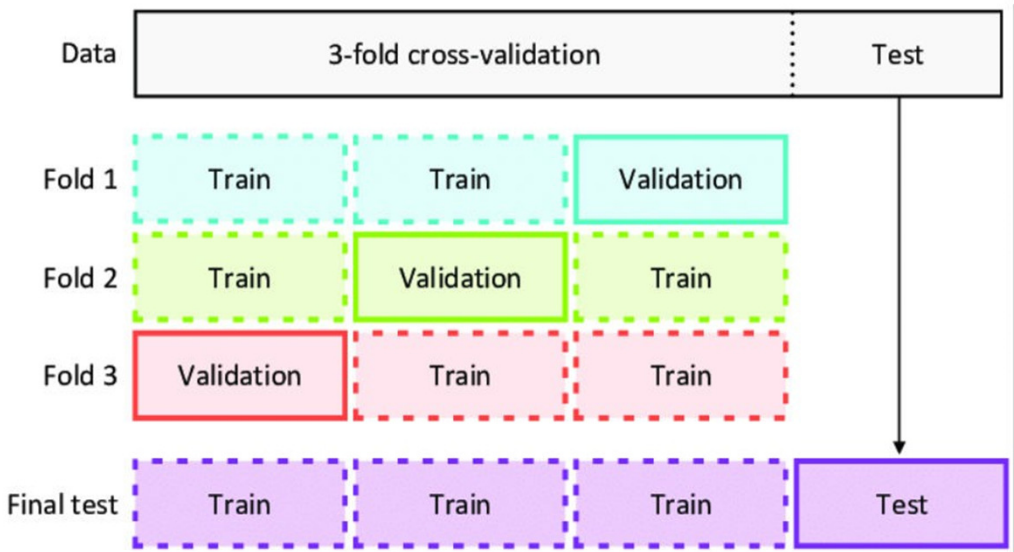
Cross Validation

For example, a 3-fold cross validation would involve training and testing a model 3 times:

1: Train on folds 1+2, test on fold 3

2: Train on folds 1+3, test on fold 2

3: Train on folds 2+3, test on fold 1



Testing Algorithms

- When starting with a problem statement and having defined a test harness, now it is time to spot check a variety of machine learning algorithms.
- Spot checking is useful because it allows you to check very quickly, if there are any learnable structures in the data and estimate which algorithms may be more effective on the problem.
- It also helps to work out any issues in the test harness and make sure that the selected performance measure is appropriate.
- The best algorithm to spot check is a random. Plug in a random number generator to generate predictions in the appropriate range. If you achieve the worst "algorithm result" and will be the measure by which all improvements can be assessed.
- Select some standard algorithms that are appropriate for your problem statement and run them through your test harness.
- Anyone standard algorithm may be appropriate for your problem, means that the algorithms can handle regression if you have a regression problem.

Testing Algorithms

- Choose methods from the algorithm. For example, user like to include a diverse mix and have 10-20 different algorithms drawn from a diverse range of algorithm types. Depending on the library used, user may spot check up to a 50+ popular methods.
- If user want to run a lot of methods, user may have to revisit data preparation and reduce the size of your selected dataset.
- This may reduce user confidence in the results, so test with various data set sizes.
- User may like to use a smaller size dataset for algorithm spot checking and a fuller dataset for algorithm tuning.

Performance Metrics

- There are various metrics which can be use to evaluate the performance of ML algorithms, **classification** as well as **regression** algorithms. One must carefully choose the metrics for evaluating ML performance because –
  - How the performance of ML algorithms is measured and compared will be dependent entirely on the metric chosen.
  - How user values the importance of various characteristics in the result will be influenced completely by the metric chosen.
- Performance Metrics is categorized as:
  - Classification Metrics
  - Regression Metrics

Classification Metrics

- Here, various performance metrics that can be used to evaluate predictions for classification problems is discussed.
  - Confusion Matrix
  - Classification Accuracy
  - Classification Report
  - AUC (Area Under ROC curve)
  - LOGLOSS (Logarithmic Loss)

Confusion Matrix

- It is the easiest way to measure the performance of a classification problem where the output can be of two or more type of classes.
- A confusion matrix is nothing but a table with two dimensions viz. “Actual” and “Predicted”.
- Furthermore, both the dimensions have “True Positives (TP)”, “True Negatives (TN)”, “False Positives (FP)”, “False Negatives (FN)”.

Confusion Matrix

Predicted \ Actual	1	0
1	**True Positives (TP)**	**False Positives (FP)**
0	**False Negatives (FN)**	**True Negatives (TN)**

- Explanation of the terms associated with confusion matrix are as follows –
- **True Positives (TP) –** It is the case when both actual class & predicted class of data point is 1.
- **True Negatives (TN) –** It is the case when both actual class & predicted class of data point is 0.
- **False Positives (FP) –** It is the case when actual class of data point is 0 & predicted class of data point is 1.
- **False Negatives (FN) –** It is the case when actual class of data point is 1 & predicted class of data point is 0.

Classification Accuracy

- It is most common performance metric for classification algorithms.
- It may be defined as the number of correct predictions made as a ratio of all predictions made.
- We can easily calculate it by confusion matrix with the help of following formula –

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

Classification Report

- This report consists of the scores of **Precisions**, **Recall**, **F1** and **Support**. They are explained as follows –
- **Precision —** ***What percent of your predictions were correct?***
- Precision is the ability of a classifier not to label an instance positive that is actually negative. For each class, it is defined as the ratio of true positives to the sum of a true positive and false positive.
- Precision:- Accuracy of positive predictions .

$$Precision = \frac{TP}{TP + FP}$$

- **Recall —** ***What percent of the positive cases did you catch?***
- Recall is the ability of a classifier to find all positive instances. For each class it is defined as the ratio of true positives to the sum of true positives and false negatives.
- Recall:- Fraction of positives that were correctly identified.

$$Recall = \frac{TP}{TP + FN}$$

Classification Report

- **F1 score —** ***What percent of positive predictions were correct?***
- This score will give us the harmonic mean of precision and recall.
- Mathematically, F1 score is the weighted average of the precision and recall.
- The best value of F1 would be 1 and worst would be 0.
- It can be calculated with the help of following formula –

$$F1\ score = \frac{2 * (precision * recall)}{(precision + recall)}$$

- **Support**
- Support is the number of actual occurrences of the class in the specified dataset.

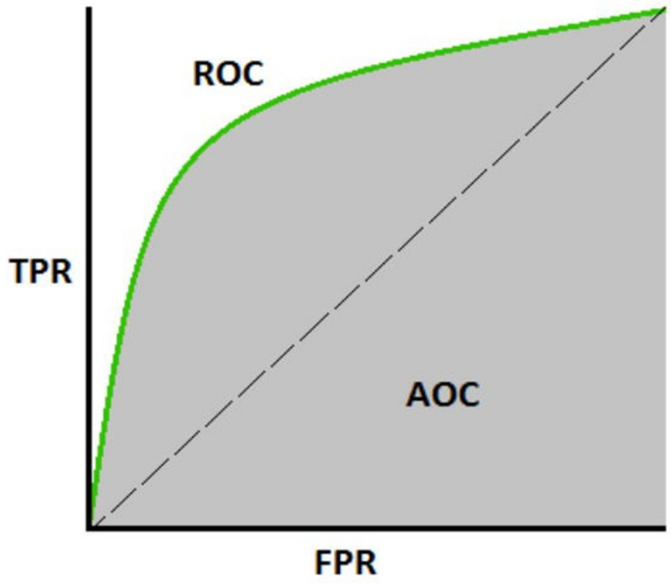
AUC (Area Under ROC curve)

- AUC (Area Under Curve)-ROC (Receiver Operating Characteristic) is a performance metric, based on varying threshold values, for classification problems.
- As name suggests, ROC is a probability curve and AUC measure the separability.
- In simple words, AUC-ROC metric will state the ability of model in distinguishing the negative and positive classes. Higher the AUC, better the model.
- ROC can be categorized into **sensitivity** and **specificity**.
  - Sensitivity is the true positive rate also called as recall. It is the number of instances from the positive(first) class that were actually predicted correctly.
  - Specificity is also called as the true negative rate. It is the number of instances from the negative (Second) class that were actually predicted correctly.

AUC (Area Under ROC curve)

Mathematically,it can be created by plotting TPR (True Positive Rate) i.e. Sensitivity or recall vs FPR (False Positive Rate) i.e. 1-Specificity, at various threshold values.

Following is the graph showing ROC, AUC having TPR at y-axis and FPR at x-axis –



LOGLOSS (Logarithmic Loss)

- It is also called Logistic regression loss or cross-entropy loss.
- It basically defined on probability estimates and measures the performance of a classification model where the input is a probability value between 0 and 1.
- It can be understood more clearly by differentiating it with accuracy.
- It is known that accuracy is the count of predictions (predicted value = actual value) in our model whereas Log Loss is the amount of uncertainty of our prediction based on how much it varies from the actual label.
- The Log Loss value will help to have more accurate view of the performance of a model.

Regression Metrics

- Here, various Regression metrics that can be used to evaluate predictions for regression problems are discussed.
  - Mean Absolute Error (MAE)
  - Mean Square Error (MSE)
  - R Squared (R^2)

Mean Absolute Error (MAE)

- It is the simplest error metric used in regression problems.
- It is basically the sum of average of the absolute difference between the predicted and actual values.
- In simple words , the MAE can gives an idea of how wrong the predictions were.
- MAE does not indicate the direction of the model i.e. no indication about underperformance or overperformance of the model. It gives the idea of magnitude of the error.
- The following is the formula to calculate MAE –

$$MAE = \frac{1}{n} \sum |Y - \hat{Y}|$$

Here, y = Actual Output Values

And $\hat{Y}$ = Predicted Output Values.

Mean Square Error (MSE)

- MSE is like the MAE, but the only difference is that the it squares the difference of actual and predicted output values before summing them all instead of using the absolute value.

- The difference can be noticed in the following equation –

$$MSE = \frac{1}{n}\sum(Y - \hat{Y})^2$$

Here, y = Actual Output Values

And $\hat{Y}$ = Predicted Output Values.

R Squared (R^2)

- R Squared metric is generally used for explanatory purpose and provides an indication of the goodness or fit of a set of predicted output values to the actual output values.
- The following formula will help us understanding it –

$$R^2 = 1 - \frac{\text{MSE(model)}}{\text{MSE(baseline)}}$$

- MSE(model): Mean Squared Error of the predictions against the actual values
- MSE(baseline): Mean Squared Error of mean prediction against the actual values

$$R^2 = 1 - \frac{\frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2}{\frac{1}{n}\sum_{i=1}^{n}(Y_i - \bar{Y}_i)^2}$$

Ensemble Methods

- Ensembles can give us a boost in the machine learning result by combining several models.
- Basically, ensemble models consist of several individually trained supervised learning models and their results are merged in various ways to achieve better predictive performance compared to a single model.
- Ensemble methods are meta-algorithms that combine several machine learning techniques into one predictive model in order to **decrease variance** (bagging), **bias** (boosting), or **improve predictions** (stacking).
- The primary principle behind the ensemble model is that a group of weak learners come together to form an active learner.

Ensemble Methods

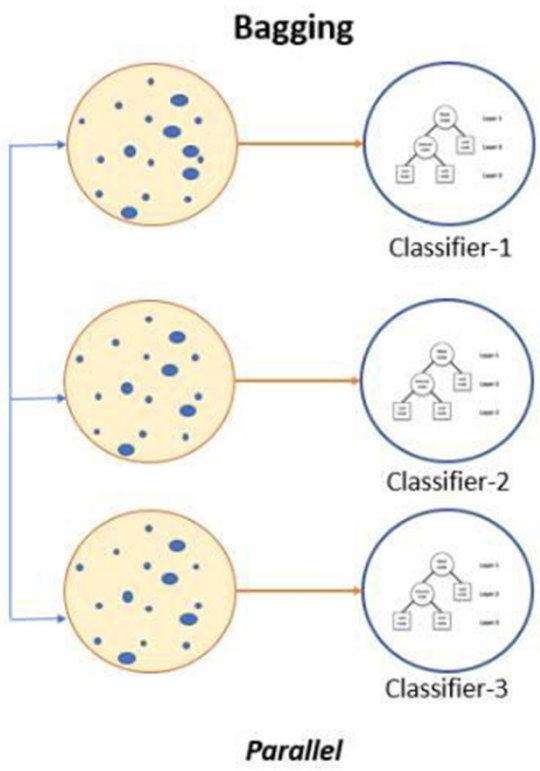
- Ensemble methods can be divided into following two groups –
- **Sequential ensemble methods**
- As the name implies, in these kind of ensemble methods, the base learners are generated sequentially. The motivation of such methods is to exploit the dependency among base learners.
- **Parallel ensemble methods**
- As the name implies, in these kind of ensemble methods, the base learners are generated in parallel. The motivation of such methods is to exploit the independence among base learners.

Ensemble Methods

- Most ensemble methods use a single base learning algorithm to produce homogeneous base learners, i.e. learners of the same type, leading to ***homogeneous ensembles***.
- There are also some methods that use heterogeneous learners, i.e. learners of different types, leading to ***heterogeneous ensembles*.**
- In order for ensemble methods to be more accurate than any of its individual members, the base learners have to be as accurate as possible and as diverse as possible.
- The techniques given below that are used to perform ensemble decision tree.
  - Bagging
  - Boosting
  - Stacking

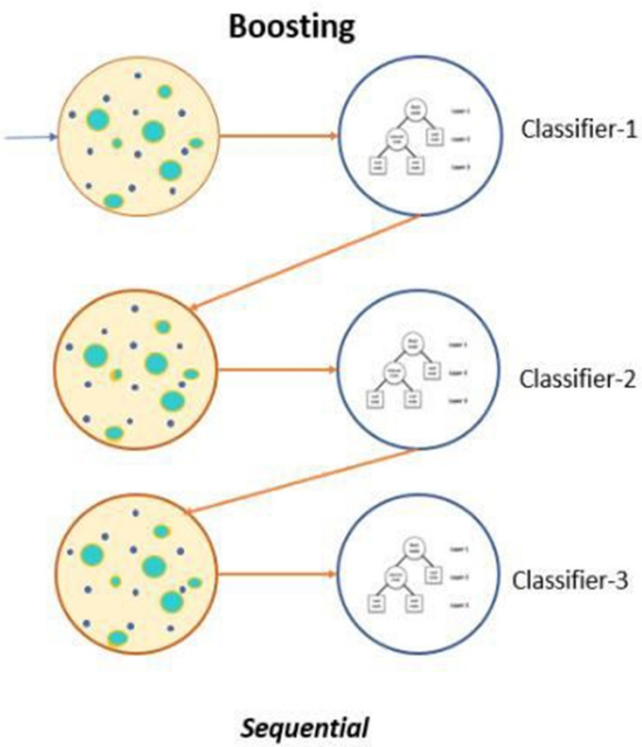
Bagging

- The term bagging is also known as **bootstrap aggregation.**
- In bagging methods, ensemble model tries to improve prediction accuracy and decrease model variance by combining predictions of individual models trained over randomly generated training samples.
- The final prediction of ensemble model will be given by calculating the average of all predictions from the individual estimators.
- One of the best examples of bagging methods are **random forests**.



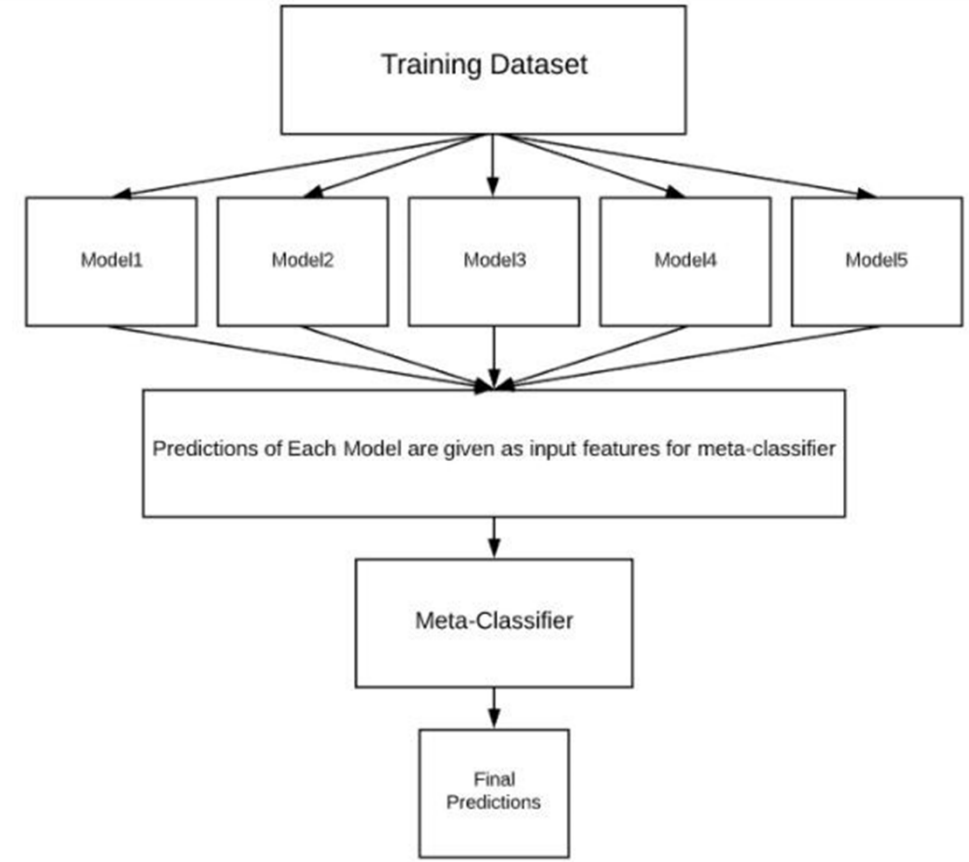
Boosting

- In boosting method, the main principle of building ensemble model is to build it incrementally by training each base model estimator sequentially.
- As the name suggests, it basically combine several weak base learners, trained sequentially over multiple iterations of training data, to build powerful ensemble.
- During the training of weak base learners, higher weights are assigned to those learners which were misclassified earlier.
- The example of boosting method is **AdaBoost.**



Stacking

- Stacking is an ensemble learning technique that combines multiple classification or regression models via a meta-classifier or a meta-regressor.
- The base level models are trained based on a complete training set, then the meta-model is trained on the outputs of the base level model as features.
- The base level often consists of different learning algorithms and therefore stacking ensembles are often heterogeneous.



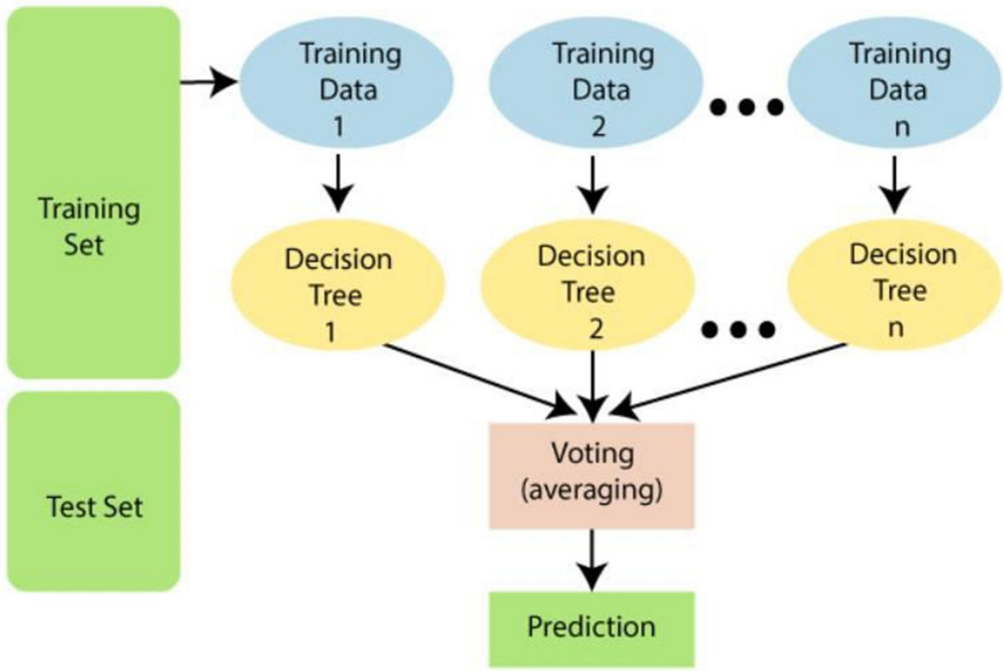
What is Random Forest?

- Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique.
- It can be used for both Classification and Regression problems in ML.
- It is based on the concept of **ensemble learning,** which is a process of *combining multiple classifiers to solve a complex problem and to improve the performance of the model.*
- As the name suggests, ***"Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset."***

What is Random Forest?

- Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.
- The **greater number of trees** in the forest **leads to higher accuracy** and **prevents** the problem of **overfitting**.

The below diagram explains the working of the Random Forest algorithm:



Assumptions for Random Forest

- Since the random forest combines multiple trees to predict the class of the dataset, it is possible that some decision trees may predict the correct output, while others may not.
- But together, all the trees predict the correct output. Therefore, below are two assumptions for a better Random forest classifier:
  - There should be some actual values in the feature variable of the dataset so that the classifier can predict accurate results rather than a guessed result.
  - The predictions from each tree must have very low correlations.

Why to use Random Forest?

- Below are some points that explain why we should use the Random Forest algorithm:
  - It takes less training time as compared to other algorithms.
  - It predicts output with high accuracy, even for the large dataset it runs efficiently.
  - It can also maintain accuracy when a large proportion of data is missing.

How does Random Forest algorithm work?

- Random Forest works in two-phase first is to create the random forest by combining N decision tree, and second is to make predictions for each tree created in the first phase.
- The Working process can be explained in the below steps and diagram:

Step-1: Select random K data points from the training set.

Step-2: Build the decision trees associated with the selected data points (Subsets).

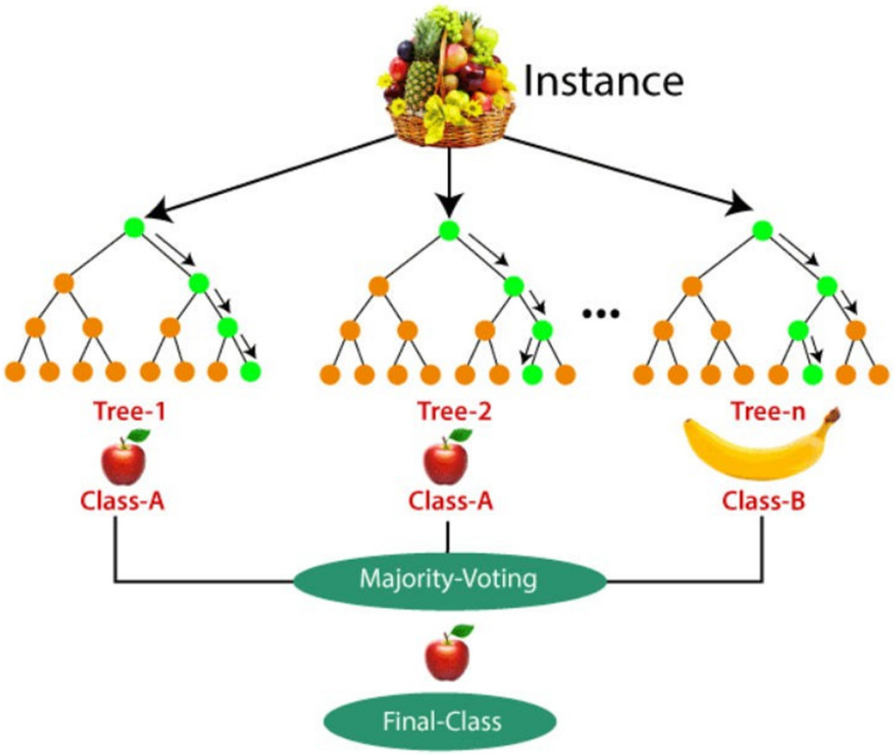
Step-3: Choose the number N for decision trees that you want to build.

Step-4: Repeat Step 1 & 2.

Step-5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

Example:

- Suppose there is a dataset that contains multiple fruit images. So, this dataset is given to the Random forest classifier. The dataset is divided into subsets and given to each decision tree. During the training phase, each decision tree produces a prediction result, and when a new data point occurs, then based on the majority of results, the Random Forest classifier predicts the final decision.



Advantages and Disadvantages of Random Forest

- **Advantages**
- Random Forest is capable of performing both Classification and Regression tasks.
- It is capable of handling large datasets with high dimensionality.
- It enhances the accuracy of the model and prevents the overfitting issue.
- **Disadvantages**
- Although random forest can be used for both classification and regression tasks, it is not more suitable for Regression tasks.

Applications of Random Forest

- There are mainly four sectors where Random forest mostly used:

  1.Banking: Banking sector mostly uses this algorithm for the identification of loan risk.

  2.Medicine: Here, disease trends and risks of the disease can be identified.

  3.Land Use: We can identify the areas of similar land.

  4.Marketing: Marketing trends can be identified.